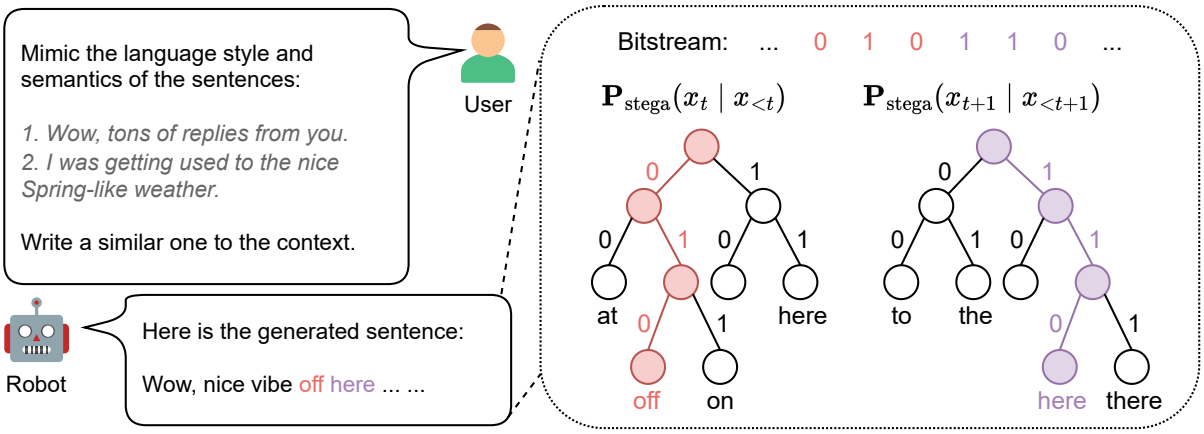

Figure 3: A running example of the embedding module and in-context stegotext generation. The selected context of samples from covertext in gray is used to instruct the stegotext generation. Huffman trees are constructed based on the conditional distribution, with the word matching the prefix of a bistream being selected as the next token. For example, the red token has 5 candidate words after applying probability pruning. Huffman tree is constructed for the 5 candidates and the word "off" matches the prefix of the bitstream and is chosen as the next word.

the extraction of secret bitstream is almost identical to the hiding process. The extraction algorithm is left out in Appendix A.2 for simplicity.

Annealing Selection. Candidate word selection with specific thresholds τ may result in trivial candidates in some cases (e.g., $|\boldsymbol{c}| = 1$). We note that trivial candidates often lead to a chain of trivial cases in subsequent stegotext generation, reducing the embedding rate and bit per word (BPW). Inspired by the stimulated annealing algorithm (van Laarhoven and Aarts, 1987), we propose the annealing search to prevent trivial candidates:

$$\mathbf{P}'_{\text{stega}}(x_t \mid x_{<t}) = \text{Softmax}(\frac{\text{LM}(x_t \mid x_{<t})}{T_t}) \quad (3)$$

Here, the temperature T_t at timestep t varies according to the previous candidates $\boldsymbol{c}_{t-1}$, the initial temperature T_0, and the annealing factor α:

$$T_t \leftarrow \begin{cases} \alpha \cdot T_{t-1} & |\boldsymbol{c}_{t-1}| = 1 \\ T_0 & |\boldsymbol{c}_{t-1}| > 1 \end{cases} \quad (4)$$

Repeat Penalty. Another corner case is that several generated words may repeat themselves after a few turns, but escape the previous annealing cases. The recently generated words are penalized for short-term repetition by applying a moving penalty:

$$\mathbf{P}''_{\text{stega}}(x_t \mid x_{<t}) = \text{Softmax}(\text{LM}(\cdot) - \boldsymbol{\delta}_t) \quad (5)$$

Penalties are based on the decay factor β, initial penalty δ_0, and the previously selected word. For any word $c \in \boldsymbol{c}_{t-1}$, $\boldsymbol{\delta}_t(c)$ has the following value:

$$\boldsymbol{\delta}_t(c) \leftarrow \begin{cases} \max\{0, \boldsymbol{\delta}_{t-1}(c) - \beta\} & c \text{ not selected} \\ \delta_0 & c \text{ selected} \end{cases} \quad (6)$$

3.3 In-Context Stegotext Generation

The core of the embedding module is to calculate the conditional probability $\mathbf{P}_{\text{stega}}(x_t \mid x_{<t})$. We use the question-answer paradigm with pre-defined prompting context C using the LLaMA2-Chat-7B (Touvron et al., 2023) as the stegotext generator. In other words, the distribution of stegotext can be represented as $\mathbf{P}_{\text{stega}}(x_t \mid x_{<t}, C)$, which is conditioned on both the previously generated tokens and the context from the covertext.

Context Selection. To approximate the distribution of covertext, we propose to incorporate several random samples from the covertext dataset into the prompting context. Since the covertext dataset is mostly available to both the sender and receiver, only a little information (e.g., random state for selection) is required to be pre-shared across different parties to reconstruct the prompting context. Therefore, an eavesdropper who suspects stegotext will be unable to decrypt the message.

In-Context Generation. As LLM is capable of comprehending the semantics of context due to its generalization ability, we utilize in-context QA tasks as a way to instruct the model to mimic the style and semantics of the covertext. To be more

specific, we design the following structured QA format: <ctx$_1$,...,ctx$_k$,[MISS]>, where the QA model is instructed to complete the missing sentence [MISS] which is similar to the context. The number of contextual sentences k is set as a hyperparameter during inference. The details of the in-context QA task can be found in Appendix A.3.

4 Experiments

4.1 Experiment Setup

Datasets. Our experiments evaluate the prior works and our method using two widely used large-scale datasets, the IMDB dataset (Maas et al., 2011) and the Twitter dataset (Go et al., 2009). The texts in each dataset are broken down into sentences by the `nltk` toolkit. Statistics are shown in Table 2.

Baselines. Since no training is needed for our zero-shot approach, we compare the performance of our method with several *training-free* methods: (a) **NLS** (Ziegler et al., 2019): This method embeds a secret bitstream using GPT-2 (Radford et al., 2019) for sentence complement. (b) **SAAC** (Shen et al., 2020): This method improves the embedding process using Self-Adjusting Arithmetic Coding.

Furthermore, we also compare our benchmarks against some *fully-supervised* steganographic techniques: (a) **RNN-Stega** (Yang et al., 2019a): This method utilizes Recurrent Neural Network models trained on covertext to generate stegotext. (b) **VAE-Stega** (Yang et al., 2021): This method adopts a Variational Auto-encoder to generate stegotext. (c) **ADG** (Zhang et al., 2021): This method proposes the Adaptive Dynamic Grouping techniques for better imperceptibility.

Implementation Details. All of the models and benchmarks are implemented with `PyTorch`. Pre-trained models (e.g., GPT-2 and LLaMA2-Chat) are obtained using `Huggingface` library. To ensure a fair comparison, we rebuild all baseline methods using the same datasets throughout the entire experiment. Bits per word (BPW) are used to measure the capability of hiding information within stegotext. Only steganographic methods with similar BPW should be compared. Additionally, we adopt a unified BPW by the `nltk` toolkit to eliminate vocabulary inconsistencies. In each setting, 8K stegotext is generated for evaluation. We adopt the LLaMA2-Chat-7B as the QA model and set the context size of our method to $k = 2$, the threshold $\tau = 0.005$, the factors $\alpha = 1.25, \beta = 0.5$, and the initial values $T_0 = 1.0, \delta_0 = 4.0$. More details of baselines can be found in Appendix A.4.

4.2 Evaluation Metrics

Perplexity. As a measure of the generation quality, the perplexity (PPL) is commonly used:

$$\text{PPL} = 2^{-\frac{1}{n}\sum_{i=1}^{n}\log_2 \mathbf{P}(x_i|x_{<i})} \tag{7}$$

JSD. Following Section 2.2, we measure the statistical imperceptibility of the covertext and stegotext using JSD. Specificly, we finetune two GPT-2 models on covertext $\mathcal{C}_{\text{cover}}$ and stegotext $\mathcal{C}_{\text{stega}}$, respectively, representing $\mathbf{P}_{\text{LM}}$ and $\mathbf{P}_{\text{stega}}$. An evaluation of the JSD is then conducted using randomly sampled text $\mathcal{C}_{\text{sample}}$. Three variants of JSD with respect to $\mathbf{P}_{\text{LM}}$ and $\mathcal{C}_{\text{sample}}$ are designed:

- *Full* JSD: Stego-models are trained on $\mathcal{C}_{\text{cover}}$ if needed[2]. $\mathcal{C}_{\text{sample}}$ are sampled from $\mathcal{C}_{\text{cover}}$. Full JSD measures the statistical imperceptibility between covertext and stegotext.
- *Half* JSD: Covertext dataset is split into two equal parts, $\mathcal{C}^1_{\text{cover}}$ and $\mathcal{C}^2_{\text{cover}}$. While the stego-models are trained on $\mathcal{C}^1_{\text{cover}}$, the evaluation is performed on $\mathcal{C}^2_{\text{cover}}$. It estimates imperceptibility under different but similar distributions.
- *Zero* JSD: Zero JSD is similar to Full JSD except that it uses a vanilla GPT-2 model as the $\mathbf{P}_{\text{LM}}$, without fine-tuning on the covertext. This evaluates the imperceptibility between the stegotext and the normal text.

However, PPL and JSDs are not good metrics for assessing different stegosystems. While PPL and JSD comparisons are suitable for supervised text generation tasks like machine translation, where a direct comparison with a golden text is possible, they face challenges in general text generation due to the absence of such direct relationships between generated output and training data. In steganography, where language distributions may vary significantly across methods, comparing PPLs and JSDs becomes impractical. Additionally, variated settings of language models, such as datasets, hyperparameters, and model architecture, will further complicate such comparisons, even if these models are within the same domain or task.

Though PPL and JSD *cannot* be used to compare generation ability among different methods, it still provides an indication of performance between different settings of the same method.

[2]Only supervised methods need training on covertext.

Methods	Training-free	IMDB					Twitter				
		BPW	PPL	JSD_{full}	JSD_{half}	JSD_{zero}	BPW	PPL	JSD_{full}	JSD_{half}	JSD_{zero}
RNN-Stega (LSTM)		1.978	10.23	30.33	33.12	38.27	2.556	13.04	39.92	38.97	48.10
		2.682	12.80	26.76	29.36	34.87	3.359	15.38	36.20	35.53	44.76
		3.351	17.02	22.66	25.72	30.28	4.139	19.78	32.17	31.75	39.19
VAE-Stega (BERT-LSTM)		1.972	9.68	34.50	36.47	38.53	2.247	10.06	46.07	45.82	46.61
		2.601	12.38	31.31	33.02	34.56	2.861	12.39	43.89	44.02	43.64
		3.199	16.31	30.03	31.49	32.82	3.438	16.13	42.12	42.54	40.87
ADG		4.931	56.22	18.24	21.19	22.86	5.702	63.86	25.92	25.35	27.68
NLS	✔	1.889	10.40	23.63	22.83	17.91	2.059	10.95	37.71	36.17	29.34
		2.531	12.90	22.08	21.28	16.73	2.806	14.01	36.61	35.17	29.45
		3.140	16.70	20.37	19.63	14.44	3.513	18.68	34.42	32.90	30.18
SAAC	✔	4.471	28.74	18.28	16.40	13.17	5.078	36.74	33.75	32.11	23.08
		4.749	37.89	18.04	16.06	11.49	5.299	43.35	33.42	31.82	22.33
		5.111	44.02	17.87	15.98	11.44	5.716	54.35	33.20	31.68	22.04
ours	✔	1.906	8.81	17.90	16.86	13.40	2.550	9.48	30.90	29.34	24.90
		2.420	13.70	18.37	17.37	13.67	3.265	14.44	30.99	29.45	25.32
		3.376	45.04	18.61	17.87	13.91	4.029	47.37	31.74	30.18	25.40

Table 1: Experimental results under different BPWs and datasets. JSDs are multiplied by 10^2. With the *same* BPW, lower JSD ↓ indicates better performance. There is no data available for smaller BPW in ADG or SAAC due to their design of high embedding rates. Our proposed method shows a reduced Psic Effect in comparison to prior methods as our JSDs rise with increasing BPW, whereas other methods see a decrease in JSDs.

Dataset	# sentences	Avg. Words	Avg. Bits
IMDB	494,716	26.70	146.43
Twitter	2,063,307	12.31	114.85

Table 2: Statistics of the preprocessed datasets.

4.3 Experimental Results

Table 1 summarizes the main results on the IMDB and Twitter datasets. Some methods are designed in such a way that it is inapplicable to adjust the BPW during sentence generation, especially for a small BPW, resulting in a lack of experiments with certain settings. This also poses a challenge for comparing metrics between similar BPWs.

Psic Effect An interesting finding in our results is the Perceptual-Statistical Imperceptibility Conflict Effect (**Psic Effect**) mentioned by Yang et al. (2021), a phenomenon in which an increase in BPW and PPL is associated with a decrease in JSD. As shown in Table 1, Figure 4, and Table 3, while techniques like NLS exhibit superior PPL and JSD scores, they produce lower-quality text and are more susceptible to detection by attackers. The existence of this unusual phenomenon implies that it will be more difficult for machines to detect stegotext when the text becomes more *chaotic* and *unintelligible*. This indicates the inconsistency of imperceptibility as perceived by humans and machines, as examples shown in Appendix C. We believe that previous methods may have fallen into some local optimum of covertext distribution, but do not achieve a balance between the quality and distribution of the stegotext.

In conclusion, the state-of-the-art metrics do not match our method in every case, but our stegotext is much more natural and **does not suffer from the Psic Effect**, as is evidenced in the case study (Fig. 4) and Section 4.5.

Furthermore, experiments with three variants of JSD have shown that our zero-shot method can be used to fool experts with specialized backgrounds and ordinary people without prior knowledge. The generalization ability of our methods ensures better performance than supervised methods in both few-shot and large-data scenarios.

4.4 Steganalysis

To assess the statistical imperceptibility of stego-methods, a classifier is usually deployed as a steganalyzer to distinguish between covertext and stegotext. Table 3 presents the results of our steganalysis. Most prior works have primarily focused on syntactic similarity, which we believe is insufficient as text evaluation should encompass both syntactic analysis and semantic analysis. Thus we utilize three distinct methods, namely TS-BiRNN (Yang